Evaluating and Enhancing Adversarial Robustness of ML Models for NIDS

IIT Kanpur B.Cyber Programme — Project Report

Hitvika Pathak | June 2026

Evidence of Work / Portfolio Summary

GitHub repository: https://github.com/Hitvikapathak/adversarial-nids-cicids2017

Local path: C:\Users\HP\adversarial-nids-cicids2017

Reproduce: python run.py

Dataset: CIC-IDS2017 — Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv

Models: Random Forest, XGBoost, MLP

Attacks: FGSM + PGD (white-box MLP, transfer to tree models)

Defenses: Adversarial training, feature squeezing, ensemble

Key result: RF clean 97.5% → PGD transfer detection 0.0%

Demo video script: docs/demo_video_script.md

Abstract

This project investigates whether machine-learning-based Network Intrusion Detection Systems remain reliable under adversarial manipulation. Using CIC-IDS2017 (Thursday Web Attacks subset, 170,366 raw flows), I trained Random Forest, XGBoost, and MLP classifiers and evaluated them with FGSM and PGD under a STRIDE threat model. High clean accuracy did not guarantee security: Random Forest dropped from 97.5% clean accuracy to 0.0% attack-detection rate under transferred PGD (ε=0.05). Adversarial training improved MLP PGD detection to 97.0%. The work shows why NIDS models must be tested under attacker-aware conditions before deployment.

Problem Statement

ML-NIDS classify network flows from statistical features. If an attacker can perturb those features within realistic bounds, malicious traffic may be labeled benign. This project measures that risk on CIC-IDS2017 and compares mitigation strategies.

Why This Matters for Cybersecurity

- Evasion attacks can bypass automated alerts in SOC pipelines.

- Clean-test accuracy hides failure modes that appear only under adversarial input.
- Robustness checks should be integrated into secure SDLC for detection systems.
- Layered defense (ML + constraints + logging + analyst review) is required in production.
- For national infrastructure, robust detection is a security control, not a leaderboard metric.

Dataset and Preprocessing

Source: CIC-IDS2017 (Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv). Raw records: 170,366; features used: 78; missing values handled: 20; duplicates removed in analysis: 6,066. Labels collapsed to binary Benign vs Attack. Class distribution (raw): {'BENIGN': 168186, 'Web Attack □ Brute Force': 1507, 'Web Attack □ XSS': 652, 'Web Attack □ Sql Injection': 21}. Preprocessing: drop identifiers, numeric coercion, top-30 variance features, StandardScaler, balanced sampling (4680 flows), split 72% train / 8% val / 20% test (stratified).

Threat Model using STRIDE

STRIDE	NIDS Example	Feature-Level Attack	Defense
Tampering	Alter timing behavior	Flow Duration / IAT perturbation	Constraint validation
Spoofing	Mimic benign traffic	Reduce packet-rate features	Behavioral baselines
Repudiation	Hide malicious footprint	Suppress volume features	Immutable flow logs
Information Disclosure	Model probing	Boundary search via queries	Access control
Denial of Service	Flood crafted flows	High-volume adversarial samples	Rate limits + robust model
Elevation of Privilege	Bypass policy action	Force benign classification	Defense-in-depth rules

Realistic Perturbation Constraints

- Packet counts and durations remain non-negative.
- Protocol identifiers are not arbitrarily changed.
- Derived statistics stay internally consistent.
- Perturbations preserve attack intent (evasion, not benign conversion).
- Categorical network fields are not treated as free continuous pixels.
- All perturbations clipped to $L\infty$ bounds after scaling.

Models Implemented

Random Forest (200 trees, depth 20, balanced class weights) for strong tabular baseline and interpretability. XGBoost (200 estimators) for boosted nonlinear boundaries. MLP (64-32 ReLU, Adam) as differentiable surrogate for gradient attacks.

Adversarial Attack Design

FGSM and PGD require gradients, so white-box attacks were executed on MLP using finite-difference gradients over attack-class loss. Random Forest and XGBoost were evaluated with transfer attacks from MLP-generated adversarial examples.

Attack	Epsilon	Steps	Step Size	Targeted?	Model
FGSM	0.01, 0.05, 0.1	1	—	No	MLP (white-box)
PGD	0.01, 0.05, 0.1	20	0.01	No	MLP (white-box)
Transfer	0.05 primary	—	—	No	RF, XGBoost

Defense Mechanisms

1) Adversarial training: MLP retrained on clean + PGD examples (ε=0.05). 2) Feature squeezing: 4-bit precision reduction before inference. 3) Ensemble: majority vote of Random Forest and robust MLP.

Experimental Setup and Reproducibility

Seed: 42. Environment: Python 3.12, Windows 10, CPU execution. Attack evaluation set: 200 attack flows. Command: python run.py. Artifacts: results/metrics.json, results/*.png, models/all_models.joblib.

Results and Graphs

PGD/FGSM columns report attack-detection rate on adversarial attack flows.

Model	Clean Acc	Clean Recall	F1	FGSM Det.	PGD Det.	Post-Defense PGD
Random Forest	97.5%	97.9%	97.4%	0.0%	0.0%	—
XGBoost	97.8%	97.7%	97.6%	0.0%	0.0%	—
MLP	94.6%	91.1%	94.0%	92.5%	92.5%	97.0%

Defense	Clean Acc	PGD Det.	Compute Cost
Baseline MLP	94.6%	92.5%	Low
Adversarial Training (MLP)	94.4%	97.0%	High (retraining)
Feature Squeezing (MLP)	94.6%	92.5%	Low
Ensemble (RF + Robust MLP)	95.9%	97.0%	Medium

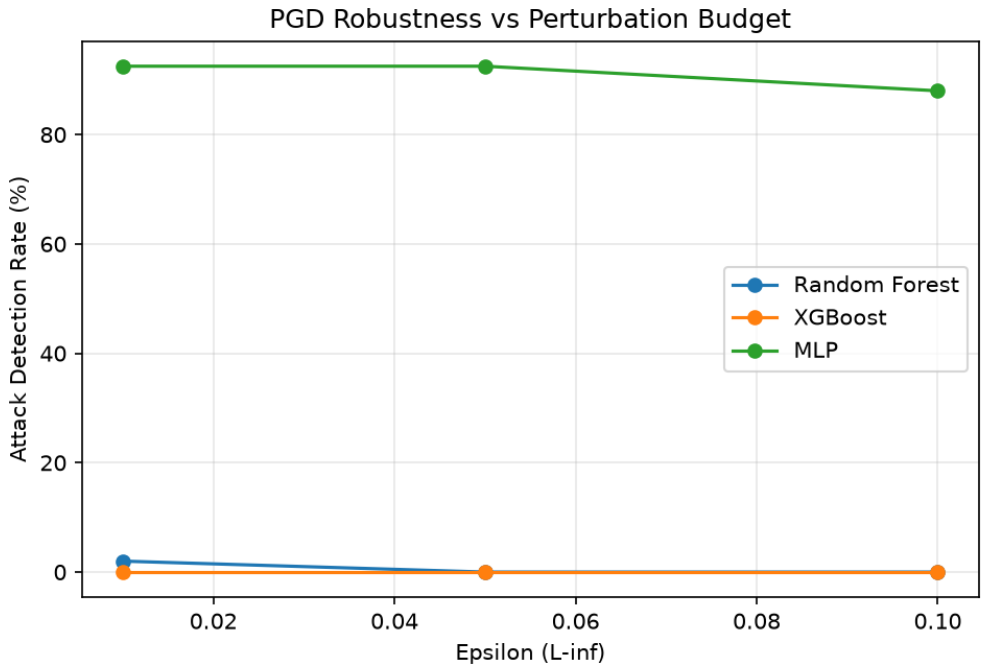


Figure 1: Attack-detection rate vs epsilon.

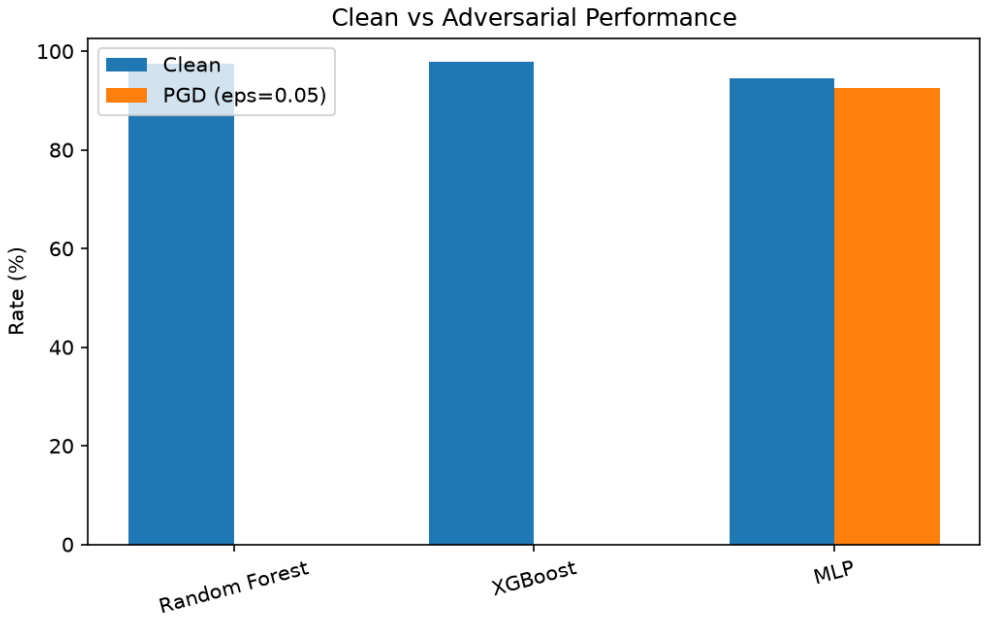


Figure 2: Clean vs PGD performance by model.

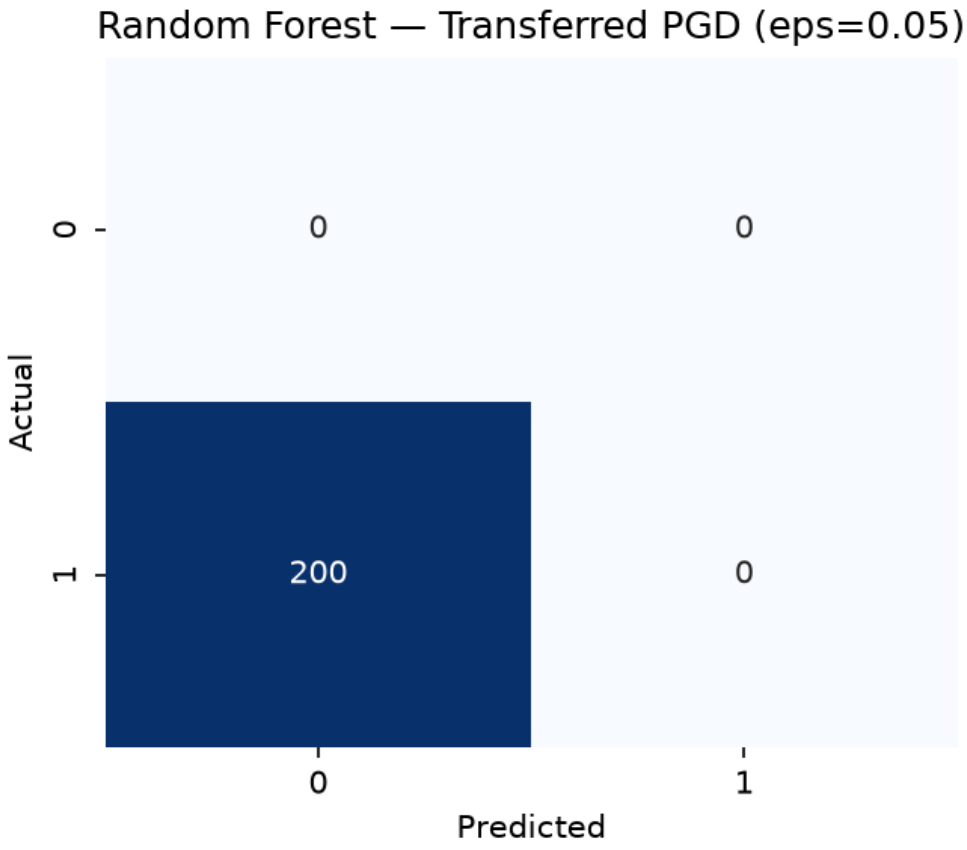


Figure 3: Random Forest under transferred PGD.

At ε=0.1, MLP detection fell to 88.0%, showing sensitivity to larger perturbation budgets.

Failure Analysis

- Transferred PGD was most damaging to tree models (0.0% RF detection).
- MLP white-box PGD at ε=0.05 retained 92.5% detection but degraded at higher epsilon.
- Feature squeezing did not improve PGD detection in this run.
- Class imbalance mitigation helped clean recall, but attack-only adversarial recall remained the critical metric.
- Adversarial training improved MLP PGD detection to 97.0% with minor clean-accuracy trade-off.

Deployment Considerations

Deploy with feature constraint checks, dual-model ensemble scoring, centralized logging, periodic red-team adversarial tests, and human analyst escalation for low-confidence alerts.

Limitations

Single-day CIC-IDS2017 file, sampled flows for compute limits, feature-level attacks (not packet-level), and finite-difference gradients instead of native autograd.

Future Work

Extension: From Model Robustness to Operational Security

- FastAPI dashboard for live flow classification.
- API security middleware for inference endpoint protection.
- Cloud deployment demo with monitored endpoints.
- Threat-analysis pipeline from traffic logs.
- CTF-style adversarial evasion challenge.

References

[1] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). Explaining and Harnessing Adversarial Examples.

[2] Madry, A., et al. (2018). Towards Deep Learning Models Resistant to Adversarial Attacks.

[3] Carlini, N., & Wagner, D. (2017). Towards Evaluating the Robustness of Neural Networks.

[4] Xu, W., Evans, D., & Qi, Y. (2018). Feature Squeezing.

[5] Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). CIC-IDS2017 dataset paper.

[6] Nicolae, I., et al. (2018). Adversarial Robustness Toolbox.

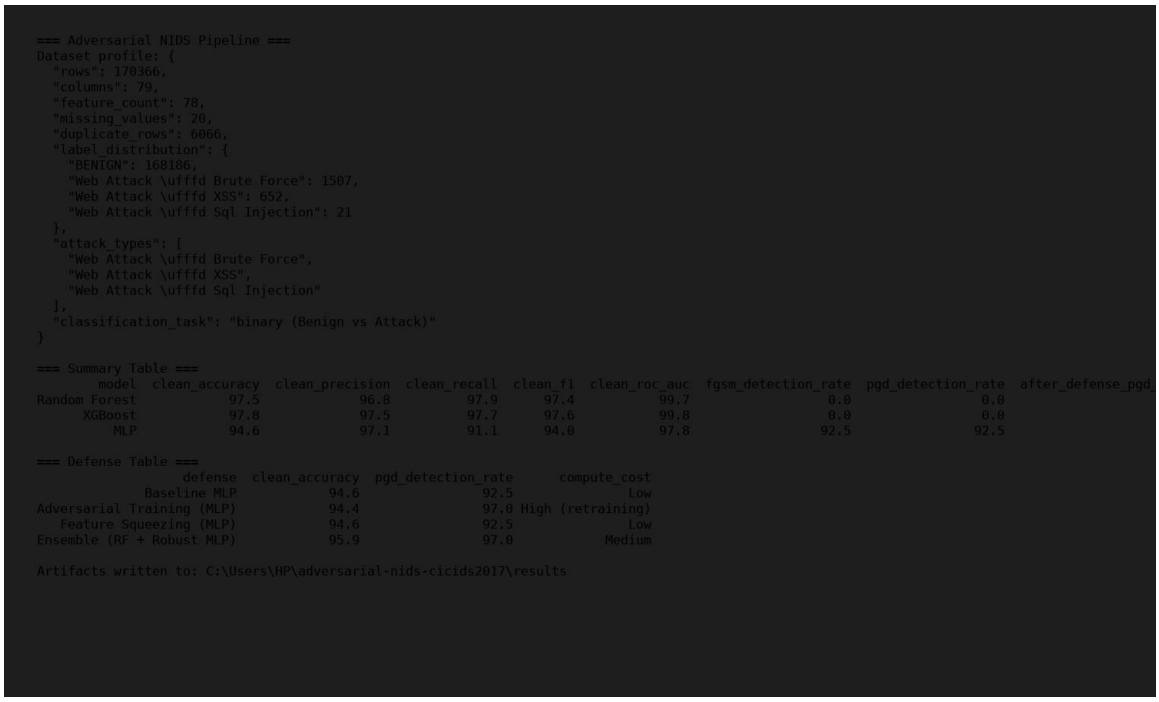
Appendix: GitHub, Screenshots, Code Snippets

GitHub: https://github.com/Hitvikapathak/adversarial-nids-cicids2017

Repository root: C:\Users\HP\adversarial-nids-cicids2017

Reproduce: python run.py

Screenshot artifact: results/screenshots/terminal_output.png



Appendix Figure: Pipeline terminal output.

Code snippet (attack call): pgd_adv = pgd_attack(mlp, attack_x, attack_y, epsilon=0.05)

Personal Reflection

I chose adversarial robustness in network intrusion detection because clean accuracy alone can create a false sense of security. During this project, I saw Random Forest reach 97.5% clean accuracy on CIC-IDS2017, then fall to 0.0% attack-detection rate under transferred PGD examples. That gap changed how I think about ML in cybersecurity: model training is only one step, and attacker-aware evaluation must be part of deployment.

At IIT Kanpur B.Cyber, I want to build operational security systems, not only offline models. My next steps are a FastAPI inference dashboard, API abuse detection around model endpoints, and a CTF-style challenge where participants craft traffic features to test detector robustness. I am

especially interested in combining threat modeling (STRIDE), red-team style testing, and practical defenses for national digital infrastructure.